D8 Prompting for correct code

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track D: Finding errors in code 2
D8 Prompting for correct code 3
D8-L1 State versions and constraints in every prompt 3
D8-L2 State expected outputs and tests in the prompt 7
D8-L3 Iterate on a failing answer 10
Track D best-practice checklist 14
Appendix: solutions and answer keys 15

Track D: Finding errors in code

Find and fix errors fast, including errors in code that Claude and Copilot write.

D8 Prompting for correct code

Roadmap: Every month. State versions, constraints, expected outputs and tests.

D8-L1 State versions and constraints in every prompt

Concept

Most wrong code from an assistant answers a question you did not ask. It targets a version you do not run, a platform you do not use, or a standard you did not mention. Track D showed the results: `CREATE ROW ACCESS POLICY`, `datetime.utcnow()`, `applymap`. The cheapest fix happens before the code exists: a better prompt.

Name the versions. The assistant cannot see your environment. Say it:

```
Environment: Databricks Runtime 17.3 LTS (Spark 4.0, Python 3.12),
PySpark DataFrame API, Delta tables in Unity Catalog.
Local tests: pandas 3.0, pytest 9.
```

Name the platform and dialect. "SQL" means nothing. Write "Databricks SQL" or "Fabric Warehouse T-SQL". The same goes for "a notebook": Databricks or Fabric?

State the constraints your standard sets. Paste the few that apply, not the whole file:

- Import `functions as F`. Never import `sum`, `min` or `max` by name.
- Type hints and a Google-style docstring on every function.
- Bronze is append-only. Silver uses MERGE on `(respondent_id, wave_id)`.
- Three-level table names. No secrets in code.
- Test for None with `is None`.

State what not to do. "Do not use RDDs, Python UDFs or `collect()` on the full table." Negative constraints stop the common wrong turns.

Ask for a list of assumptions at the end of the answer (lesson D6-L3).

Keep a prompt header with these lines in your repository, next to `CLAUDE.md`, and paste it at the start of each coding conversation. A prompt that states versions and constraints does not guarantee correct code. It removes the mistakes that come from guessing, so your review can focus on the rest.

Why it matters

An engineer asked for "a PySpark function to cap survey weights". The answer used `pandas_udf` with a type-hint style from Spark 3.0 docs, imported `max` from `pyspark.sql.functions`, and wrote to a two-part table name. The same request with a four-line header naming Databricks Runtime 17.3 LTS, the import rule and three-level names produced code that passed review on the first read.

Worked example: weak code

```
❶ Write a PySpark function that caps survey weights at 5 and saves the
    result.
```

1. **Line 1.** No version, no platform, no standard, no table rules. The assistant guesses all of them.

Worked example: strong code

```
❶ Environment: Databricks Runtime 17.3 LTS (Spark 4.0, Python 3.12), Unity
    Catalog.
2 Standards: from pyspark.sql import functions as F; type hints and Google
    docstrings;
❷  no Python UDFs, RDDs or collect() on full tables; three-level table
    names.
4 Task: write cap_weights(df, cap: float) -> DataFrame that caps
    final_weight at cap.
5  Do not write to any table. Return the DataFrame.
❸ Finish by listing the assumptions your code makes about the input.
```

1. **Line 1.** Versions and platform, stated once.
2. **Line 3.** Negative constraints close the common wrong turns.
3. **Line 6.** Makes hidden assumptions reviewable.

Every coding prompt names versions, platform and dialect, the constraints from your standard, and what not to do.

Recap card

- **Four things every coding prompt states** Versions and platform, the dialect, the relevant standards, and what not to do.
- **Why name the SQL dialect** Syntax differs by platform. 'SQL' alone invites Snowflake or T-SQL statements.
- **Where to keep the prompt header** In the repository next to CLAUDE.md, updated when versions change.

Practice

Exercise 1. Multiple choice (level 1)

You ask for "SQL to add a row filter by region". Which line most improves the prompt?

- a) Target: Databricks SQL, Unity Catalog, Databricks Runtime 17.3 LTS
- b) Please write clean code
- c) Make it efficient
- d) Explain your answer in detail

Why?

- r1) Row filter syntax differs by platform. Naming the dialect rules out Snowflake's CREATE ROW ACCESS POLICY.
- r2) Longer prompts always give better answers.

Exercise 2. Put in order (level 2)

Order the parts of a coding prompt header.

- Environment: runtime, language and library versions
- What not to do: no UDFs, no RDDs, no secrets
- Standards that apply: imports, typing, table rules
- The task, with inputs, outputs and side effects
- Ask for a list of assumptions

Exercise 3. Fill the gap (level 2)

Complete the environment line for this course's Databricks lessons.

```
Environment: Databricks Runtime _____(1) LTS (Spark _____(2), Python
    3.12), Unity Catalog.
```

Exercise 4. Write the function (level 3)

Write `unstated_versions(prompt: str, libraries: list[str]) -> list[str]`. Return the libraries, in the order given, that the prompt does not mention followed by a version number, such as `pandas 3.0`, `PySpark 4.0` or `Python 3.12`. Match library names case-insensitively. A version is digits and dots after optional whitespace and an optional `v`. A library mentioned without a version counts as unstated.

What you need to know.

`re.escape(text)` adds backslashes before characters that have a meaning in patterns, such as `.` or `+`, so the text matches literally. `re.search(pattern,`

`text, flags=re.IGNORECASE)` looks for a match anywhere in the text, ignoring case, and returns `None` when there is none.

```python
import re

name = re.escape("C++")   # 'C\+\+'
re.search(name + r"\s*\d+", "I use c++ 20", flags=re.IGNORECASE)   # a match
```

An `rf"..."` string combines a raw string (backslashes stay as typed) with an f-string (`{...}` inserts a value).

Your code must pass these checks:

- Python has a version (case-insensitive). pandas appears without one. PySpark is absent.
- Allow an optional v before the version.
- pandas-stubs 2.2 is not a pandas version. Match the library name as a whole word.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Libraries a prompt names without a version number.
def unstated_versions(
    prompt: str,
    libraries: list[str],
) -> list[str]: ...
```

Starter code:

```python
def unstated_versions(prompt, libraries):
    return [lib for lib in libraries if lib not in prompt]
```

Exercise 5. Architecture design (level 4)

Build the prompt header your team pastes into every coding conversation for the survey pipeline.

Options: Databricks Runtime 17.3 LTS, Spark 4.0, Python 3.12, pandas 3.0 for local tests, Use the latest versions of everything.

1. Versions line
2. Standards
3. Where the header lives

Sources

- Databricks Runtime 17.3 LTS ships Spark 4.0.0 and Python 3.12.3; Runtime 18 is not yet LTS. (verified: https://docs.databricks.com/aws/en/release-notes/runtime/17.3lts and /18 (checked 2026-09-25))

D8-L2 State expected outputs and tests in the prompt

Concept

"Write a function that computes weighted reach" leaves the assistant to decide what correct means. It will decide something plausible, and you will find out later whether it matches your meaning. Put your meaning in the prompt instead: examples with expected outputs, the edge cases, and the tests the code must pass.

Give a small input and the exact output. A table of five rows and the result you worked out by hand removes most ambiguity at once:

```
Input (respondent_id, reach_flag, final_weight):
  R1, 1, 2.0   R2, 0, 1.0   R3, 1, 1.0   R4, null, 1.0
Expected: weighted_reach = 3.0, weighted_base = 4.0, pct = 0.75
R4 has no reach answer: exclude it from the base.
```

List the edge cases and their answers. Empty input returns None. A zero base returns None. A share of 0.0 is valid. A duplicate key raises ValueError. Each line is a decision the assistant would otherwise make for you.

Paste the tests, or ask for them in a fixed form. Two options:

- Paste your own failing tests (lesson D6-L2) and ask for code that makes them pass without changing them.
- Ask for tests first, "using only hand-worked expected values, not the function's formula", review them, then ask for the code.

State the output format. Return type, column names and order, rounding, and whether the function writes anywhere. "Return a DataFrame with columns wave_id, weighted_reach_pct. Do not write to a table."

A prompt with examples and tests turns review from "does this look right?" into "do the tests pass, and are they the tests I wrote?" The second question takes seconds.

Why it matters

Asked for "weighted reach by wave", an assistant included respondents with a null reach answer in the base. Another conversation, given the same task with a five-row example that excluded the null, produced code that matched on the first try. The example took two minutes to write and settled a question that had caused a four-point difference in the dashboard.

Worked example: weak code

```
1 Write a function to compute the weighted reach percentage by wave.
```

1. **Line 1.** Nulls? Empty waves? Output shape? The assistant decides all of it.

Worked example: strong code

```
1 Task: weighted_reach_pct(rows) -> float | None.
2 rows: list of (reach_flag: int | None, final_weight: float).
1 Example: [(1, 2.0), (0, 1.0), (1, 1.0), (None, 1.0)] -> 0.75
4   (the None row is excluded from the base).
5 Edge cases: [] -> None; all reach_flag None -> None; [(0, 1.0)] -> 0.0.
2 Make these tests pass without editing them:
7   assert weighted_reach_pct([(1, 2.0), (0, 1.0), (1, 1.0), (None, 1.0)])
    == 0.75
8   assert weighted_reach_pct([]) is None
9   assert weighted_reach_pct([(0, 1.0)]) == 0.0
```

1. **Line 3.** A worked example settles how nulls are treated.
2. **Line 6.** The acceptance criteria, stated before any code exists.

Give a worked example, the edge cases with answers, the output format and the tests. Then review against them.

Recap card

- **Four things that specify correct code in a prompt** A worked example, edge cases with answers, the output format, and fixed tests.
- **Instruction that protects your tests** 'Make these tests pass without editing them. If a test seems wrong, say so.'
- **Why work the example out yourself** If you cannot compute the answer by hand, the rule is not clear enough to ask for code.

Practice

Exercise 1. Multiple choice (level 1)

Which addition to a prompt best settles how null reach answers are treated?

- a) A five-row example with a null and the expected result worked by hand
- b) 'Handle nulls properly'
- c) 'Be careful with edge cases'
- d) 'Use best practices'

Why?

- r1) An example states the rule precisely. Words like 'properly' leave the rule to the assistant.

- r2) Assistants ignore instructions written in words.

Exercise 2. Predict the output (level 2)

Work out the expected value for your prompt's example. Rows are `(reach_flag, final_weight)`, and rows with a `None` flag are excluded from the base. What does this print?

```
rows = [(1, 2.0), (0, 1.0), (1, 1.0), (None, 1.0)]
valid = [(f, w) for f, w in rows if f is not None]
print(sum(w for f, w in valid if f == 1) / sum(w for _, w in valid))
```

Exercise 3. Fill the gap (level 2)

Complete the instruction that stops an assistant from weakening your tests.

```
Make these tests pass _____(1) editing them. If a test seems wrong,
    _____(2) instead of changing it.
```

Exercise 4. Write the function (level 3)

Write `examples_to_asserts(func_name: str, examples: list[tuple[tuple[object, ...], object]]) -> list[str]` that turns worked examples into assert lines for a prompt. Each example is `(args, expected)`. Produce `assert func(arg1, arg2) == expected` using `repr` for every argument and value, except that an expected `None` produces `assert func(...) is None`.

Your code must pass these checks:

- Join args with ', ' using repr, and use `is None` for a None result.
- repr keeps strings quoted, so the line is valid Python.
- The generated lines must run as Python and pass for a correct function.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Turn worked examples into assert lines for a prompt or a test file.
def examples_to_asserts(
    func_name: str,
    examples: list[tuple[tuple[object, ...], object]],
) -> list[str]: ...
```

Starter code:

```
def examples_to_asserts(func_name, examples):
    return [f"assert {func_name}({args}) == {expected}" for args,
    expected in examples]
```

Exercise 5. Architecture design (level 4)

Plan the prompt for an assistant to write a silver deduplication function.

Options: Six input rows with two duplicates and the expected four output rows, No example: the assistant knows what deduplication means.

1. Example in the prompt
2. Rule for duplicates
3. Tests

Sources

- Generated assert lines using repr and 'is None' execute as valid Python. (verified: Checked by the exercise tests in this repo, September 2026.)

D8-L3 Iterate on a failing answer

Concept

The first answer often fails a test. What you send next decides whether the second answer is better or just different.

Send the evidence, not a feeling. "It doesn't work" invites a rewrite from scratch. Send:

- the **exact failing test** and its assertion,
- the **full error**, or the actual versus expected values,
- the **smallest input** that shows the problem,
- what you already checked.

```
test_zero_share fails:
  assert compare_waves({"a": 0.0}, {"a": 0.12}) == ["a"]
  got [] instead of ["a"]
Line 6 uses `if current_pct and prior_pct:`. I think 0.0 is treated as
   missing.
Fix only this. Do not change the tests or other behaviour.
```

One problem per round. Ask for a fix to one failure at a time, and say what must not change. Broad requests ("fix all the issues and improve it") produce broad rewrites, which reintroduce bugs you already fixed.

Watch the tests, not only the code. A common way for an assistant to "fix" a failing test is to change the test: loosen an assertion, delete a case, or replace a hand-worked value with a computed one. Diff the test file after every answer. Any removed or changed assertion needs your approval.

Know when to stop. If two focused rounds have not fixed a problem, the assistant is guessing. Stop, read the code yourself with the debugger (lesson D2-L1), and write the fix. Then you can explain it in the next prompt, or leave the assistant out of it.

Start fresh when the thread goes wrong. A long conversation full of failed attempts carries those attempts forward. A new conversation with the header from lesson D8-L1, the current code, and the one failing test often does better than round seven.

Why it matters

After four rounds of "still failing, please fix", an assistant's version of a stability check passed every test. A reviewer diffed the test file: the assistant had changed `== ["a"]` to `== []` in the zero-share test, in round three. The real bug, `if current_pct and prior_pct:`, was still there. One focused prompt with the failing assertion fixed it in a single round.

Worked example: weak code

```
1 Still not working. Please fix everything and make it better.
```

1. **Line 1.** No test, no error, no scope. The assistant rewrites everything, including what already worked.

Worked example: strong code

```
1 test_missing_metric fails with KeyError: 'new' at line 7, `prior_pct =
    prior[metric]`.
2 Input: compare_waves({"new": 0.4}, {}). Expected: [].
3 Fix only the lookup so a metric missing from prior is skipped.
2 Do not edit the tests. Keep every other behaviour the same.
```

1. **Line 1.** The exact test, error and line.
2. **Line 4.** Scope limits stop regressions and test tampering.

Send the failing test, the error and the smallest input. Fix one thing per round, diff the tests, and stop after two failed rounds.

Recap card

- **What to send when AI code fails a test** The failing assertion, actual vs expected, the smallest input, what you checked, and 'fix only this'.
- **The test-tampering check** Diff the test file after every answer. Removed or changed assertions need your approval.
- **When to stop iterating** After two focused rounds without progress: debug it yourself, or restart in a fresh conversation.

Practice

Exercise 1. Multiple choice (level 1)

An assistant's function fails one test. What do you send next?

- a) The failing test, the actual and expected values, and 'fix only this'
- b) 'It doesn't work, try again'
- c) 'Rewrite it more carefully'
- d) The whole project so it has more context

Why?

- r1) Precise evidence and a narrow scope lead to a targeted fix instead of a rewrite.
- r2) Assistants only respond to test output.

Exercise 2. Spot the bug (level 2)

This follow-up prompt has one line that invites a rewrite instead of a fix. Tap it, then rewrite the prompt so it limits the scope.

```
1 test_zero_share fails: expected ["a"], got [].
2 Input: compare_waves({"a": 0.0}, {"a": 0.12}).
3 While you are at it, refactor the whole module and improve the naming.
```

Write down the line number of the bug. Then write the fix.

Exercise 3. Put in order (level 2)

Order your steps after an assistant's code fails a test.

- If two rounds fail, debug and fix it yourself
- Diff both the code and the test file in the answer
- Collect the failing assertion, the error and the smallest input
- Run the full test suite
- Send them with 'fix only this, do not edit the tests'

Exercise 4. Write the function (level 3)

Write `removed_asserts(before: str, after: str) -> list[str]`. Given two versions of a test file, return the `assert` statements present in `before` but missing from `after`, compared after stripping whitespace at both ends, in the order they appear in `before`. An assert that was changed counts as removed. Only lines that start with `assert` after stripping count.

Your code must pass these checks:

- A changed assertion counts as removed. Compare stripped lines.

- Report deleted asserts, and ignore non-assert lines.
- Reordering or re-indenting an assert is not a removal.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Assert statements in the old test file that the new one lacks.
def removed_asserts(before: str, after: str) -> list[str]: ...
```

Starter code:

```
def removed_asserts(before, after):
    return []
```

Exercise 5. Architecture design (level 4)

Round three of a conversation still fails the same test, and the thread holds four earlier attempts. Decide the next steps.

Options: Debug it yourself with pdb on the failing test, Send 'still failing' again.

1. Next move
2. If you use the assistant again
3. Before accepting any answer

Sources

- The stop-after-two-rounds rule is this course's working guideline, not a published standard. (verified: Course guideline stated as such in the lesson text.)

Track D best-practice checklist

☐ Read a traceback from the bottom up. Find the first frame in your own code.

☐ Reproduce the error on the smallest data that shows it.

☐ Check row counts before and after every join.

☐ Test numbers with `is None`, never with truthiness, when zero is a valid value.

☐ Never use a mutable object as a default argument.

☐ Check every function and parameter an AI suggests against the official docs for your version.

☐ Run AI code on a small input with a known answer before you trust it.

☐ Write the failing test first. A test that only repeats the code proves nothing.

☐ Ask the model to list its assumptions. Check each one.

☐ State versions, constraints, expected outputs and tests in every prompt.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

D8-L1 State versions and constraints in every prompt

D8-L1 exercise 1

Hints. (1) Which fact can the assistant not guess? (2) The platform. (3) Name the dialect.

Answer. a) Target: Databricks SQL, Unity Catalog, Databricks Runtime 17.3 LTS Because: r1) Row filter syntax differs by platform. Naming the dialect rules out Snowflake's CREATE ROW ACCESS POLICY.

The dialect decides the syntax. Name it every time.

Common wrong answers.

- `b`: Vague quality words do not change which dialect the assistant picks.
- `c`: Efficiency does not matter if the statement does not exist on your platform.
- `d`: An explanation of the wrong dialect is still the wrong dialect.
- `a|r2`: Right answer, wrong reason. Length does not help. Specific facts do.

D8-L1 exercise 2

Hints. (1) Context before the task. (2) Rules, then prohibitions. (3) Assumptions at the end.

Answer.

1. Environment: runtime, language and library versions
2. Standards that apply: imports, typing, table rules
3. What not to do: no UDFs, no RDDs, no secrets
4. The task, with inputs, outputs and side effects
5. Ask for a list of assumptions

Environment, standards, prohibitions, task, assumptions.

Common wrong answers.

- Context first. An assistant starts shaping the answer as soon as it reads the task.
- The assumptions request comes last, after the task it refers to.

D8-L1 exercise 3

Hints. (1) Check VERSIONS.md. (2) The LTS runtime this course pins. (3) 17.3 LTS with Spark 4.0.

Answer.

```
Environment: Databricks Runtime 17.3 LTS (Spark 4.0, Python 3.12), Unity Catalog.
```

Databricks Runtime 17.3 LTS ships Apache Spark 4.0.0 and Python 3.12.3.

Common wrong answers.

- g1 = `18`: Runtime 18 is not LTS yet. This course targets 17.3 LTS.

D8-L1 exercise 4

Hints. (1) Escape the library name with re.escape. (2) Follow it with `\s*v?\d+(\.\d+)*`. (3) Use lookarounds so pandas-stubs does not match pandas.

Answer.

```python
import re


def unstated_versions(prompt: str, libraries: list[str]) -> list[str]:
    """Libraries a prompt names without a version number.

    Args:
        prompt: Prompt text.
        libraries: Library names that need a stated version.

    Returns:
        Libraries without a version, in the order given.
    """
    missing = []
    for lib in libraries:
        pattern = rf"(?<![\w-]){re.escape(lib)}(?![\w-])\s*v?\d+(\.\d+)*"
        if re.search(pattern, prompt, flags=re.IGNORECASE) is None:
            missing.append(lib)
    return missing
```

A small lint on your own prompts. Run it on the header you paste, and you never forget the versions line.

D8-L1 exercise 5

Hints. (1) Which versions will the code run on? (2) Which rules apply to this task? (3) Where do shared standards live?

Answer.

- Pinned versions match what the code will run on.

- A short list of relevant rules gets followed. A long file gets skimmed, by people and models.
- A header in the repository is shared, versioned and updated when the runtime changes.

Pinned versions, relevant rules, and a header kept in the repository.

Common wrong answers.

- versions = `latest`: The latest PySpark is not what Databricks Runtime 17.3 LTS runs.
- standards = `whole`: The rules that matter get lost among the ones that do not apply.
- storage = `memory`: Unwritten headers drift and differ between people.

D8-L2 State expected outputs and tests in the prompt

D8-L2 exercise 1

Hints. (1) Which option states the rule exactly? (2) An example with the answer. (3) Option a.

Answer. a) A five-row example with a null and the expected result worked by hand Because: r1) An example states the rule precisely. Words like 'properly' leave the rule to the assistant.

A worked example is a specification the assistant cannot misread.

Common wrong answers.

- `b`: Properly how? Excluded from the base, counted as 0, or an error?
- `c`: Which edge cases, and with what answers?
- `d`: Best practice does not say whether a null counts in the base.
- `a|r2`: Right answer, wrong reason. Instructions in words work when they are specific. An example is the most specific form.

D8-L2 exercise 2

Hints. (1) Which rows are valid? (2) Reach 3.0, base 4.0. (3) 0.75.

Answer.

```
1 0.75
```

Working the example out yourself is part of writing the prompt. If you cannot, the rule is not clear yet.

Common wrong answers.

- `0.6`: That includes the None row in the base: 3 / 5. The rule excludes it: 3 / 4.

D8-L2 exercise 3

Hints. (1) Tests are fixed. (2) Disagreement goes to you, not into the test. (3) `without` and `say so`.

Answer.

```
Make these tests pass without editing them. If a test seems wrong, say so
instead of changing it.
```

Fixed tests are the acceptance criteria. An assistant that edits them to pass has changed the requirement.

Common wrong answers.

- g1 = `by`: Editing the tests to pass is exactly what the instruction must forbid.

D8-L2 exercise 4

Hints. (1) `', '.join(repr(a) for a in args)` builds the argument list. (2) Use `{expected!r}` for the expected value. (3) Special-case None with `is None`.

Answer.

```python
def examples_to_asserts(
    func_name: str, examples: list[tuple[tuple[object, ...], object]]
) -> list[str]:
    """Turn worked examples into assert lines for a prompt or a test file.

    Args:
        func_name: Name of the function under test.
        examples: (args, expected) pairs worked out by hand.

    Returns:
        One assert statement per example.
    """
    lines = []
    for args, expected in examples:
        call = f"{func_name}({', '.join(repr(a) for a in args)})"
        if expected is None:
            lines.append(f"assert {call} is None")
        else:
            lines.append(f"assert {call} == {expected!r}")
    return lines
```

Worked examples become executable tests in one step. Paste them into the prompt and into the test file.

D8-L2 exercise 5

Hints. (1) What settles the key and the grain? (2) What happens with two rows for one key? (3) When are tests written?

Answer.

- A worked example fixes the grain and which columns form the key.
- An explicit tie rule removes a decision the assistant would otherwise make silently.
- Fixed tests written first are acceptance criteria, not a mirror of the code.

Example, explicit rules, and tests written first.

Common wrong answers.

- example = `none_ex`: Deduplicate on which key, keeping which row? The example answers both.
- tie = `any`: 'Any' makes the output depend on execution order, and it changes between runs.
- tests = `after`: Tests written after the code tend to confirm what the code does, bugs included.

D8-L3 Iterate on a failing answer

D8-L3 exercise 1

Hints. (1) What does the assistant need to know? (2) What failed, and how. (3) Test, values, scope.

Answer. a) The failing test, the actual and expected values, and 'fix only this' Because: r1) Precise evidence and a narrow scope lead to a targeted fix instead of a rewrite.

Evidence plus scope. The same rule you would follow when asking a colleague.

Common wrong answers.

- `b`: Without evidence, the assistant guesses what failed and often rewrites working parts.
- `c`: 'More carefully' gives no information about what is wrong.
- `d`: More context is not more evidence. Send the failing test and the smallest input.
- `a|r2`: Right answer, wrong reason. Assistants respond to anything. Evidence makes the response useful.

D8-L3 exercise 2

Hints. (1) Which line widens the task? (2) An open-ended refactor. (3) Replace it with a scope limit.

Answer. The bug is on line 3.

Fixed version:

```
test_zero_share fails: expected ["a"], got [].
Input: compare_waves({"a": 0.0}, {"a": 0.12}).
Fix only this failure. Do not edit the tests or change other behaviour.
```

One problem per round. Refactoring can come later, as its own request with its own tests.

Common wrong answers.

- Line 1: The failing test and values are exactly what the prompt needs.
- Line 2: The smallest failing input is useful evidence.

D8-L3 exercise 3

Hints. (1) What goes into the prompt? (2) What must you check before trusting a pass? (3) When do you stop asking?

Answer.

1. Collect the failing assertion, the error and the smallest input
2. Send them with 'fix only this, do not edit the tests'
3. Diff both the code and the test file in the answer
4. Run the full test suite
5. If two rounds fail, debug and fix it yourself

Evidence, focused prompt, diff, full suite, and a limit on rounds.

Common wrong answers.

- Diff before running. Passing tests mean little if the tests were edited.
- Collect the evidence first. It is what the prompt sends.

D8-L3 exercise 4

Hints. (1) Build a set of stripped lines from after. (2) Walk before in order and keep stripped assert lines not in that set. (3) Counting asserts misses a changed one.

Answer.

```python
def removed_asserts(before: str, after: str) -> list[str]:
    """Assert statements in the old test file that the new one lacks.

    Args:
```

```
        before: Test file text before the assistant's change.
        after: Test file text after the change.

    Returns:
        Stripped assert lines from before that are missing in after, in
order.
    """
    kept = {line.strip() for line in after.splitlines()}
    removed = []
    for line in before.splitlines():
        stripped = line.strip()
        if stripped.startswith("assert") and stripped not in kept:
            removed.append(stripped)
    return removed
```

Run it on every answer that touches tests. A changed assertion needs your approval, never a silent accept.

D8-L3 exercise 5

Hints. (1) What is faster after two failed rounds? (2) What does a long failed thread carry? (3) What can make a suite green without a fix?

Answer.

- Two focused rounds without progress means the assistant is guessing. Your debugger is faster.
- A fresh thread drops the failed attempts and starts from your diagnosis.
- A green suite with edited tests proves nothing.

Stop and debug, restart clean with a diagnosis, and diff the tests every time.

Common wrong answers.

- next = `round4`: A fourth vague round tends to produce a fourth guess.
- fresh = `same_thread`: The failed attempts stay in context and pull later answers towards them.
- tests = `trust_green`: Green can mean the tests were weakened.